



BÀI 03: TRANSACTION (GIAO DỊCH), NGOẠI LỆ TRONG JDBC

- ❑ Transaction (giao dịch)
- ❑ Ngoại lệ trong JDBC
- ❑ Xây dựng các lớp Entity, Interface DAO, các lớp DAO
- ❑ Bài tập



TRANSACTION (GIAO DỊCH)

- ❑ Mặc định, sau khi mỗi câu lệnh SQL được thực thi qua JDBC, dữ liệu sẽ được cập nhật ngay vào CSDL. (autoCommit = true)
- ❑ Có những trường hợp, ta muốn dữ liệu chỉ được cập nhật vào CSDL sau khi một số câu lệnh SQL được thực hiện:
Ví dụ: Đối với trang ứng dụng bán hàng qua mạng, để CSDL được thống nhất, ta chỉ muốn lưu các dữ liệu liên quan tới một đơn đặt hàng cùng một lúc.
- ❑ Một nhóm các câu lệnh như thế được gọi là một giao dịch (transaction)



TRANSACTION (GIAO DỊCH)

- ❑ Giao dịch là một tập hợp các thao tác SQL thực hiện liên tục để hoàn thành một công việc nhất định, và các thao tác này phải được thực hiện hoàn toàn (commit) hoặc bị hủy bỏ hoàn toàn (rollback) nếu có lỗi xảy ra.
- ❑ Đặc điểm của Giao dịch trong JDBC

Một giao dịch trong JDBC phải đảm bảo 4 tính chất gọi là **ACID**:

- ✓ **Atomicity (Nguyên tử hóa)**: Các thao tác trong giao dịch phải được thực hiện tất cả hoặc không có thao tác nào được thực hiện.
- ✓ **Consistency (Tính nhất quán)**: Giao dịch phải đưa cơ sở dữ liệu từ trạng thái nhất quán này sang trạng thái nhất quán khác.
- ✓ **Isolation (Tính biệt lập)**: Các thao tác trong một giao dịch không được phép ảnh hưởng đến các thao tác của giao dịch khác đang thực thi.
- ✓ **Durability (Tính bền vững)**: Sau khi giao dịch được hoàn thành (commit), các thay đổi sẽ được lưu lại và duy trì kể cả khi có sự cố.



TRANSACTION (GIAO DỊCH)

- ❑ JDBC hỗ trợ quản lý giao dịch thông qua việc sử dụng các phương thức `commit()` và `rollback()` của đối tượng `Connection`.
- ❑ Để thực hiện một giao dịch nhiều bước, ta cần tắt chế độ auto-commit bằng cách gọi `connection.setAutoCommit(false)`.



TRANSACTION (GIAO DỊCH)

❑ Các bước Thực hiện Giao dịch trong JDBC

1. **Bắt đầu giao dịch:** Tắt chế độ auto-commit để bắt đầu một giao dịch nhiều bước.
2. **Thực hiện các thao tác SQL:** Thực hiện một hoặc nhiều câu lệnh SQL.
3. **Commit hoặc Rollback:**
 - **Commit:** Nếu tất cả các câu lệnh thực thi thành công, gọi commit() để lưu thay đổi.
 - **Rollback:** Nếu xảy ra bất kỳ lỗi nào, gọi rollback() để hủy bỏ các thay đổi.



TRANSACTION (GIAO DỊCH)

```
9 public class Example1 {
10     public void transaction() throws SQLException{
11         Connection conn=JDBCUtil.getConnection();
12         String insertQuery = "INSERT INTO PRICELIST (name,price) VALUES(?,?);";
13         if (conn != null){
14             try{
15                 conn.setAutoCommit(false);
16                 PreparedStatement prest = conn.prepareStatement(insertQuery);
17
18                 prest.setString(1, "Biscuit");
19                 prest.setDouble(2, 1.2);
20                 prest.executeUpdate();
21
22                 prest.setString(1, "Pen");
23                 prest.setDouble(2, 0.5);
24                 prest.executeUpdate();
25                 prest.close();
26                 conn.commit();
27                 conn.setAutoCommit(true);
28             } catch (SQLException e) {
29                 e.printStackTrace();
30             }
31         }
32     }
```



TRANSACTION (GIAO DỊCH)

```
30 public void rollbackTransaction() throws SQLException{
31     Connection conn=JDBCUtil.getConnection();
32     String insertQuery = "INSERT INTO PRICELIST (name,price) VALUES(?,?);";
33     if (conn != null){
34         conn.setAutoCommit(false);
35         PreparedStatement prest =conn.prepareStatement(insertQuery);
36         prest.setString(1, "Biscuit 456");
37         prest.setDouble(2, 1.2);
38         prest.executeUpdate();
39         //conn.commit();
40         prest.setString(1, "Pen 456");
41         prest.setDouble(2, 0.5);
42         prest.executeUpdate();
43         prest.close();
44         //conn.rollback();
45         conn.commit();
46         conn.setAutoCommit(true);
47
48     }
49 }
```



TRANSACTION (GIAO DỊCH)

VD: Thực hiện thao tác trên nhiều bảng và mối quan hệ một nhiều

```
11 public class CTHD {
12
13     private int sohd;
14     private String masp;
15     private int sl;
16
17     public CTHD() {}
18
19
20     public CTHD(int sohd, String masp, int sl) {}
21
22
23
24
25
26     public void setSohd(int sohd) {
27         this.sohd = sohd;
28     }
29
30     public void setMasp(String masp) {
31         this.masp = masp;
32     }
33 }
```

```
16 public class HoaDon {
17     private int sohd;
18     private String makh, manv;
19     private Date nghd;
20     private int giatri;
21     private ArrayList<CTHD> cthd;
22
23     public HoaDon() {}
24
25
26     public HoaDon(int sohd, Date nghd, String makh, String manv, int giatri) {}
27
28
29
30
31
32
33
34
35     public HoaDon(int sohd, Date nghd, String makh, String manv, int giatri, ArrayList<CTHD>
36
37
38
39
40
41
42
43
44     public void addCT(CTHD ct) {
45         cthd.add(ct);
46     }
47 }
```



TRANSACTION (GIAO DỊCH)

Phương thức thêm mới một bản ghi HoaDon

```
public void insert(HoaDon ob, Connection con) {
    PreparedStatement pr = null;
    String sqlInsert = "Insert into hoadon (sohd,nghd,makh,manv,trigia) values (?, ?, ?, ?, ?)";

    if (con != null) {
        try {
            con.setAutoCommit(false);
            pr = con.prepareStatement(sqlInsert);
            pr.setInt(1, ob.getSohd());
            pr.setDate(2, ob.getNghd());
            pr.setString(3, ob.getMakh());
            pr.setString(4, ob.getManv());
            pr.setInt(5, ob.getGiatri());
            System.out.println(pr.toString());
            pr.executeUpdate();
            pr.close();
            DAOCTHD CT = new DAOCTHD();
            CT.insertbyshd(ob.getCthd(), con);
            con.commit();

            con.setAutoCommit(true);

        } catch (SQLException e) {

            e.printStackTrace();
        }
    }
}
```



TRANSACTION (GIAO DỊCH)

Phương thức thêm mới danh sách “chi tiết hóa đơn” CTHD

```
public void insertbyshd(ArrayList<CTHD> listct, Connection con) {  
    PreparedStatement pr = null;  
    String sqlInsert = "Insert into cthd (sohd,masp,sl) values (?, ?, ?)";  
    if (con != null) {  
        try {  
            pr = con.prepareStatement(sqlInsert);  
            for (CTHD ct : listct) {  
                System.out.println(ct.toString());  
                pr.setInt(1, ct.getSohd());  
                pr.setString(2, ct.getMasp());  
                pr.setInt(3, ct.getSl());  
                System.out.println(pr.toString());  
                pr.executeUpdate();  
            }  
            pr.close();  
        } catch (SQLException e) {  
            e.printStackTrace();  
        } finally {  
            // ConnectionMySQLUtils.closeConnec(con);  
        }  
    }  
}
```



TRANSACTION (GIAO DỊCH)

Thực hiện Rollback data khi 1 trong các câu lệnh thực thi SQL gặp ngoại lệ

```
        con.commit(); // 2. commit data to database if all command are success
    } catch (Exception e) {
        e.printStackTrace();
        con.rollback(); // 2. roll-back data if one of command are failed
        System.out.println("Rollback data");
    }
}
```



TRANSACTION (GIAO DỊCH)

- ❑ Các lưu ý khi quản lý giao dịch trong JDBC
 - ✓ **Tắt auto-commit:** Luôn đảm bảo rằng bạn tắt auto-commit trước khi bắt đầu một giao dịch nhiều bước. Sau khi giao dịch hoàn thành hoặc rollback, bạn có thể bật lại auto-commit.
 - ✓ **Kiểm tra lỗi kỹ lưỡng:** Khi thao tác với nhiều bước trong một giao dịch, cần xử lý kỹ từng bước để tránh mất dữ liệu hoặc sai sót trong các phép tính.
 - ✓ **Sử dụng try-catch-finally:** Đảm bảo đóng kết nối và tài nguyên trong finally, và thực hiện rollback trong khối catch nếu có lỗi xảy ra.



TRANSACTION (GIAO DỊCH)

- ❑ Ưu điểm của Quản lý Giao dịch trong JDBC
 - ✓ Đảm bảo tính nhất quán: Quản lý giao dịch đảm bảo rằng dữ liệu trong cơ sở dữ liệu luôn nhất quán khi có các thao tác cập nhật nhiều bước.
 - ✓ Phục hồi lỗi: Nếu có lỗi xảy ra trong một giao dịch, có thể thực hiện rollback để quay lại trạng thái ban đầu.
 - ✓ Kiểm soát hiệu quả: Người lập trình có thể kiểm soát chính xác khi nào các thay đổi nên được lưu lại vào cơ sở dữ liệu hoặc hủy bỏ nếu không cần thiết.



NGOẠI LỆ TRONG JDBC

❑ Các Ngoại lệ Phổ biến trong JDBC

- ✓ `SQLException`: Xảy ra khi cú pháp câu lệnh SQL không đúng. Thường gặp khi truy vấn hoặc thao tác với bảng.
- ✓ `SQLTimeoutException`: Phát sinh khi một thao tác vượt quá thời gian giới hạn.
- ✓ `SQLIntegrityConstraintViolationException`: Xảy ra khi có vi phạm về ràng buộc dữ liệu, chẳng hạn như vi phạm khóa chính.
- ✓ `SQLDataException`: Lỗi do không hợp lệ về định dạng dữ liệu hoặc độ dài.
- ✓ `SQLNonTransientConnectionException`: Lỗi kết nối không thể khắc phục, chẳng hạn khi máy chủ cơ sở dữ liệu không hoạt động.



NGOẠI LỆ TRONG JDBC

❑ Quản lý Ngoại lệ (Exception Handling)

- ✓ Quản lý ngoại lệ trong JDBC là một phần quan trọng để đảm bảo ứng dụng Java có thể hoạt động ổn định với cơ sở dữ liệu. Sử dụng đúng cách các loại ngoại lệ và xử lý chuỗi ngoại lệ sẽ giúp mã nguồn dễ bảo trì và giảm thiểu lỗi khi thao tác với cơ sở dữ liệu.



NGOẠI LỆ TRONG JDBC

❑ Quản lý Ngoại lệ (Exception Handling)

- ✓ Trong JDBC, nên sử dụng các khối try-catch-finally để quản lý ngoại lệ. Khối finally giúp đảm bảo rằng các tài nguyên như kết nối, statement, và result set được đóng đúng cách ngay cả khi có ngoại lệ xảy ra.
- ✓ Trong giao dịch (transaction), nếu một ngoại lệ xảy ra, bạn nên gọi rollback() để đảm bảo dữ liệu không bị lỗi hoặc không nhất quán.



XÂY DỰNG CÁC LỚP ENTITY, INTERFACE DAO, CÁC LỚP DAO

❑ interface DAO:

- ✓ interface DAO để định nghĩa các phương thức chung cho các lớp DAO khác, bạn sẽ có một cấu trúc chuẩn hóa và dễ mở rộng hơn. Interface DAO này sẽ chứa các phương thức CRUD cơ bản mà mọi lớp DAO đều cần triển khai. Điều này giúp chuẩn hóa cách bạn tương tác với dữ liệu và dễ dàng thay đổi hoặc mở rộng các lớp DAO khác nhau mà không ảnh hưởng đến các phần khác của ứng dụng.



XÂY DỰNG CÁC LỚP ENTITY, INTERFACE DAO, CÁC LỚP DAO

- ❑ **Lớp thực thể:** Lớp thực thể là lớp đại diện cho thông tin của một sinh viên, và tùy theo ngữ cảnh và cách đặt tên trong các dự án khác nhau, lớp này có thể được gọi bằng các tên khác nhau:
 - ✓ DTO (Data Transfer Object). **DTO** là mẫu thiết kế được dùng để truyền dữ liệu giữa các lớp hoặc các tầng khác nhau trong ứng dụng.
 - ✓ Trong mô hình MVC (Model-View-Controller), lớp đại diện cho dữ liệu thường được gọi là "**Model**". Vì vậy, SinhVien có thể được đặt tên là Model để thể hiện vai trò của nó trong kiến trúc MVC.
 - ✓ Trong các ứng dụng dùng ORM (Object Relational Mapping) như Hibernate, lớp đại diện cho bảng trong cơ sở dữ liệu thường được gọi là "**Entity**". Tên SinhVienEntity thể hiện rằng đây là một thực thể liên kết trực tiếp với một bảng trong cơ sở dữ liệu.
 - ✓ **POJO** (Plain Old Java Object) là thuật ngữ chỉ các lớp Java đơn giản, không phụ thuộc vào bất kỳ framework nào. SinhVienPOJO có thể là tên gọi phù hợp khi lớp không có các phụ thuộc đặc biệt và chỉ chứa dữ liệu.



XÂY DỰNG CÁC LỚP ENTITY, INTERFACE DAO, CÁC LỚP DAO

❑ JDBCUtil :

- ✓ JDBCUtil (hoặc DatabaseHelper): Lớp tiện ích dùng để quản lý kết nối với cơ sở dữ liệu.

❑ SinhVienDAO:

- ✓ SinhVienDAO: Lớp cung cấp các phương thức để thao tác với cơ sở dữ liệu như thêm, sửa, xóa, và truy vấn dữ liệu của bảng SinhVien.



CÁC THÀNH PHẦN CỦA JDBC

❑ Bài tập thực hành

Bài 1: Xây dựng các lớp cho bài toán quản lý bán hàng với các yêu cầu sau:

```
CREATE TABLE Product (  
    productID INT PRIMARY KEY,  
    name VARCHAR(100),  
    stock INT  
);  
  
CREATE TABLE Order (  
    orderID INT PRIMARY KEY,  
    productID INT,  
    quantity INT,  
    FOREIGN KEY (productID) REFERENCES Product(productID)  
);
```



CÁC THÀNH PHẦN CỦA JDBC

- ✓ Thêm vài sản phẩm vào bảng Product với số lượng tồn kho khác nhau.

Tạo Lớp ProductDAO và OrderDAO:

- ✓ Viết lớp ProductDAO với các phương thức:
 - checkStock(int productID): Kiểm tra số lượng tồn kho của sản phẩm.
 - updateStock(int productID, int quantity): Cập nhật số lượng tồn kho của sản phẩm.
- ✓ Viết lớp OrderDAO với phương thức addOrder(int orderID, int productID, int quantity): Thêm đơn hàng mới.

Quản lý Giao Dịch:

- ✓ Viết lớp TransactionExample có phương thức createOrder(int orderID, int productID, int quantity).
- ✓ Phương thức này sẽ thực hiện các bước:
 - Kiểm tra số lượng tồn kho của sản phẩm. Nếu số lượng tồn kho ít hơn quantity, hiển thị thông báo lỗi và không tạo đơn hàng.
 - Nếu đủ số lượng tồn kho, thêm một bản ghi vào bảng Order và trừ số lượng tương ứng trong bảng Product.

Sử dụng Transaction:

- ✓ Đảm bảo rằng nếu có lỗi trong quá trình cập nhật tồn kho hoặc thêm đơn hàng, toàn bộ giao dịch sẽ bị hủy bỏ bằng rollback().
- ✓ Sử dụng commit() để xác nhận giao dịch nếu thành công.



CÁC THÀNH PHẦN CỦA JDBC

❑ Bài tập thực hành

Bài 2: Xây dựng một ứng dụng Java quản lý đơn hàng và cập nhật kho, bao gồm các lớp entity, interface DAO, lớp `implement_DAO`, và lớp `ConnectUtil` để kết nối cơ sở dữ liệu.

Yêu cầu:

- ✓ Tạo cơ sở dữ liệu ShopDB với hai bảng Product và Order, trong đó bảng Product lưu thông tin sản phẩm và số lượng tồn kho, còn bảng Order lưu thông tin các đơn hàng.
- ✓ Tạo lớp `ConnectUtil` để quản lý kết nối cơ sở dữ liệu, hai lớp entity Product và Order đại diện cho các bảng tương ứng.
- ✓ Xây dựng interface `ProductDAO` với các phương thức thực hiện thao tác với CSDL Product và các phương thức `checkStock` để kiểm tra tồn kho và `updateStock` để cập nhật số lượng tồn kho, interface `OrderDAO` với phương thức `addOrder` để thêm đơn hàng. Triển khai các phương thức này trong lớp `ProductDAOImpl` và `OrderDAOImpl`.
- ✓ Sau đó, xây dựng lớp `TransactionExample` để thực hiện việc thêm đơn hàng và cập nhật kho bằng cách sử dụng transaction nhằm đảm bảo nếu xảy ra lỗi trong quá trình thêm đơn hàng hoặc cập nhật kho, toàn bộ giao dịch sẽ bị hủy bỏ (rollback) để đảm bảo dữ liệu không bị ảnh hưởng.
- ✓ Tạo lớp Main để thử nghiệm việc tạo đơn hàng mới và kiểm tra kết quả trong các tình huống kho không đủ hàng và kho đủ hàng, nhằm đảm bảo giao dịch hoạt động đúng cách.



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM